

# Julia, a programming language for Operations Research

Prof. Marc Sevaux

Lab-STICC – Université de Bretagne-Sud – Lorient – FRANCE

with the support of Dr. Alexandru OLTEANU, Quentin PERRACHON and Owein THUILLIER

April 27-29, 2026



echo \$(whoami)

<http://people.univ-ubs.fr/marc.sevaux/>

- studied in the **Applied Mathematics Institute** (Angers, France)
  - got a **Master and PhD degree** from Paris VI university, France
  - **associate prof.** 6 years at univ. of Valenciennes, France
  - since 2005, **professor** at univ. Bretagne Sud (Lorient, France)
  - **sabbatical** 2010/2011 in Helmut Schmidt univ. (Hamburg, Germany)
  - **EURO President 2020-2023** <http://www.euro-online.org>
- research topics: heuristics, metaheuristics, MP, matheuristics
  - interest: combining metaheuristics and machine learning
  - recent interest: Quantum Operations Research
  - applications: Routing, Scheduling, WSN, Sonar networks, Warehouse management, ...

# Teaching

## Teaching programs

- Mathematical engineering
- Electronic design
- Production management
- Industrial engineering

## Level of students

- Master students
- Engineer students
- PhD students

## Courses

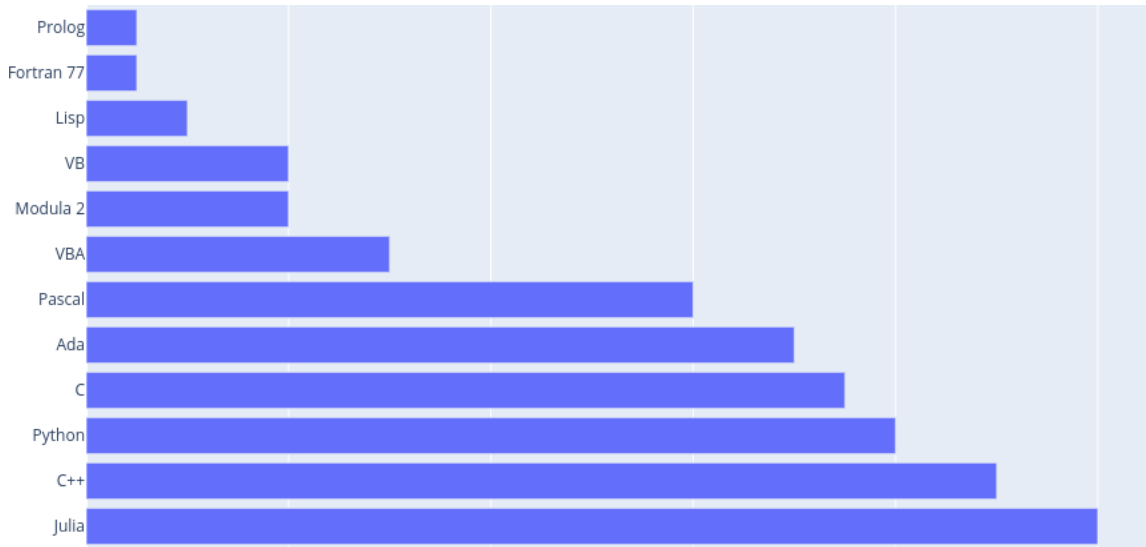
- Intro OR/graph theory
- Mathematical programming
- Heuristics/metaheuristics
- Data analysis/management
- Large scale optimization

## Type of students

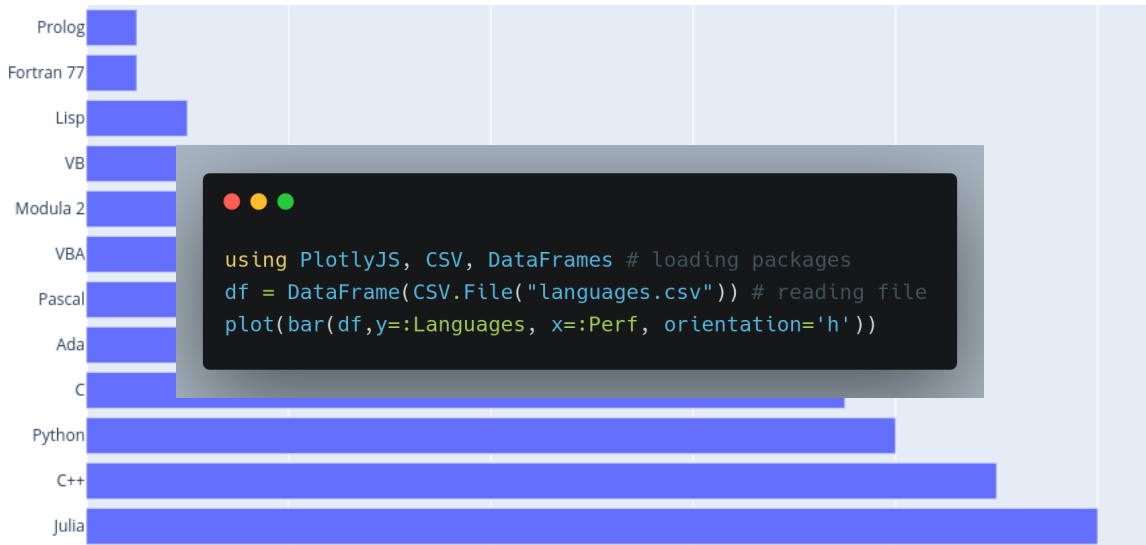
- Classical students
- Full distance learning
- Professional students

Not a specialist in Computer Science, but use programming languages in every class...

## Programming history (proficiency)



# Programming history (proficiency)



# Why Julia?

## 4 reasons to learn julia today

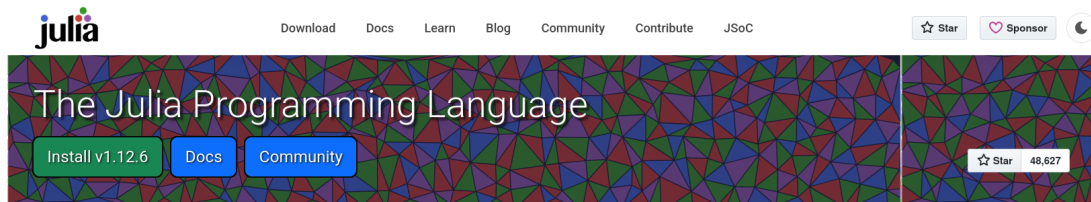
**Speed** as general as Python, as statistics-friendly as R, but as fast as C

**Syntax** dynamically-typed, closer to R and Python, easy to learn

**Multiple dispatch** Julia functions' ability to behave differently based on the types of arguments

**OR friendly** Julia's environment is well adapted to OR with rapid prototyping, speed, packages, link to other languages, ...





## Fast

Julia was designed for [high performance](#). Julia programs automatically compile to efficient native code via LLVM, and support [multiple platforms](#).

## Composable

Julia uses [multiple dispatch](#) as a paradigm, making it easy to express many object-oriented and [functional](#) programming patterns. The talk on the [Unreasonable Effectiveness of Multiple Dispatch](#) explains why it works so well.

## Dynamic

Julia is [dynamically typed](#), feels like a scripting language, and has good support for [interactive](#) use, but can also optionally be separately compiled.

## General

Julia provides [asynchronous I/O](#), [metaprogramming](#), [debugging](#), [logging](#), [profiling](#), a [package manager](#), and the ability to [build binaries](#).

## Reproducible

[Reproducible environments](#) make it possible to recreate the same Julia environment every time, across platforms, with [pre-built binaries](#).

## Open source

Julia is an open source project with over 1,000 contributors. It is made available under the [MIT license](#). The [source code](#) is available on GitHub. Julia has a [welcoming community](#) accessible to all backgrounds.

## Julia REPL (Read Evaluate Print Loop)

```
$:~ > julia
```

A 3D grid of nodes connected by edges. The nodes are arranged in a 3x3x3 cube. The top layer has nodes colored blue, red, green, and purple. The bottom layer has nodes colored blue, red, green, and purple. A path is highlighted in red, starting from the bottom-left node and ending at the top-right node.

Documentation: <https://docs.julialang.org>

```
Type "?" for help, "]?" for Pkg help.
```

Version 1.12.6 (2026-04-09)

Official <https://julialang.org> release

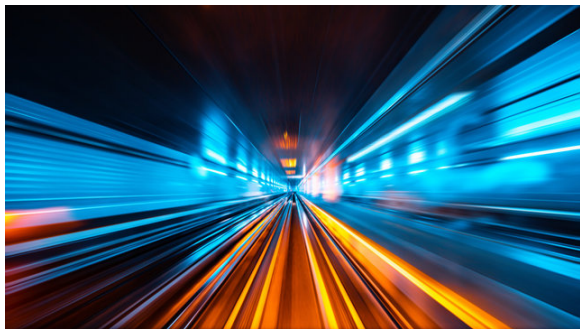
```
julia>
```



# Julia is Fast

## Quick version

You call a function, Julia will specialize that call all the way down to its concrete types (thanks to multiple dispatch).



Let's see how Julia compare to other languages...

# Julia is easy to understand

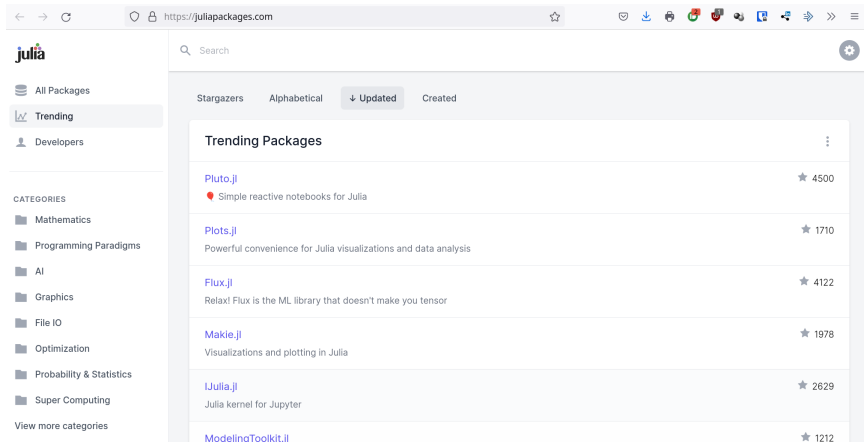


## Intuitive and easy

- Julia is fast
- Julia is easier and cleaner than C++
- You can avoid the two-language problem

See dataframes, arrays/vectors, functions and broadcast...

# Julia packages



The screenshot shows the Julia Packages website at <https://juliapackages.com>. The left sidebar contains navigation links: All Packages, Trending (selected), and Developers. Below these are categories: Mathematics, Programming Paradigms, AI, Graphics, File IO, Optimization, Probability & Statistics, and Super Computing. The main content area displays 'Trending Packages' sorted by 'Updated'. The packages listed are:

| Package                            | Description                                                     | Stars |
|------------------------------------|-----------------------------------------------------------------|-------|
| <a href="#">Pluto.jl</a>           | Simple reactive notebooks for Julia                             | 4500  |
| <a href="#">Plots.jl</a>           | Powerful convenience for Julia visualizations and data analysis | 1710  |
| <a href="#">Flux.jl</a>            | Relax! Flux is the ML library that doesn't make you tensor      | 4122  |
| <a href="#">Makie.jl</a>           | Visualizations and plotting in Julia                            | 1978  |
| <a href="#">IJulia.jl</a>          | Julia kernel for Jupyter                                        | 2629  |
| <a href="#">ModelingToolkit.jl</a> |                                                                 | 1212  |

Let's explore the packages **Graphs**, **PlotlyJS** and **JuMP**...

# Conclusion

## Cons.

- Julia is a young language
- Compile time latency may happen
- Ecosystem is immature
- Subtle syntax tricks

## Pros.

- Julia is easy to learn
- Julia REPL is amazing
- Julia is fast
- Multithreading is easy
- I will you during the week :-)